



বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

CRAG — Corrective Retrieval-Augmented Generation

সম্পূর্ণ বাংলায় লেখা বিস্তারিত নোট | তোমার নিজের কোড থেকে শেখা

১. 🌀 সহজ ভাষায় পরিচিতি (Intuition)

CRAG কী?

কল্পনা করো তুমি একটি পরীক্ষার প্রশ্নের উত্তর খুঁজছ। তুমি লাইব্রেরিতে গিয়ে বই খুঁজলে — কিন্তু বইয়ে সঠিক উত্তর নেই। এখন তুমি কী করবে?

একটি **বোকা RAG সিস্টেম** যা পাবে তাই নিয়ে উত্তর দেবে — ভুল হলেও।

কিন্তু তুমি যদি **স্মার্ট** হও, তুমি ভাববে:

- "এই বইয়ের তথ্য কি আমার প্রশ্নের সাথে মিলছে?"
- "না মিললে Google করব" (Web Search)
- তারপর মিলিয়ে উত্তর দেব।

এটাই হলো **CRAG — Corrective RAG**।

বাস্তব জীবনের উদাহরণ

🔍 রান্নার উদাহরণ:

- তুমি রেসিপি বই দেখলে (RAG: Vector Store থেকে retrieve)
- বইটি পুরনো, "এয়ার ফ্রায়ার" এর রেসিপি নেই (Evaluation: INCORRECT)
- তুমি YouTube এ সার্চ করলে (Web Search: Tavily)
- YouTube এর ভিডিও দেখে রান্না করলে (Generate: সঠিক উত্তর)

কোন সমস্যা সমাধান করে?

সমস্যা	সাধারণ RAG	CRAG
রিড্রিভ করা document irrelevant হলে?	ভুল উত্তর দেয়	ধরে ফেলে, Web Search করে
Document partially relevant হলে?	সব নিয়ে নেয়	শুধু relevant অংশ রাখে
প্রশ্নটি অনেক নতুন/আধুনিক হলে?	উত্তর দিতে পারে না	Web Search থেকে আনে

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

CRAG-এর পুরো Flow

User Question



[retrieve_node] → Vector Store থেকে Top-K document আনো



[doc_eval_node] → প্রতিটি document LLM দিয়ে evaluate করো



Score-এর ভিত্তিতে verdict দাও:
CORRECT / INCORRECT / AMBIGUOUS



CORRECT



INCORRECT / AMBIGUOUS



[refine_node]



[rewrite_query_node]



[web_search_node]



→ [refine_node]



[generate_node]



Final Answer

প্রতিটি Node কী করে?

● Node 1: retrieve_node

- Vector Store থেকে প্রশ্নের সাথে মিলে এমন Top-4 document নিয়ে আসে
- কোনো evaluation নেই — শুধু raw retrieve

● Node 2: doc_eval_node

- প্রতিটি document LLM-কে দিয়ে score করায় (0.0 থেকে 1.0)

• তিনটি verdict:

- **CORRECT**: কোনো একটি score > 0.7 হলে → সরাসরি refine এ যাও
- **INCORRECT**: সব score < 0.3 হলে → web search করো
- **AMBIGUOUS**: মিশ্র scores → web search করো (route করে refine এ যাওয়ার আগে)

● Node 3: rewrite_query_node (শুধু INCORRECT/AMBIGUOUS তে)

- মূল প্রশ্নটিকে একটি ভালো **web search query**-তে রূপান্তর করে
- উদাহরণ: "neural network কী?" → "neural network definition machine learning keywords"

🌐 Node 4: `web_search_node` (শুধু INCORRECT/AMBIGUOUS তে)

- Tavily Search API ব্যবহার করে web থেকে তথ্য আনে
- Document হিসেবে format করে রাখে

🟢 Node 5: `refine_node`

- প্রতিটি document-কে ছোট ছোট sentence-এ ভাঙে (`chunks_splitter`)
- `filter_chain` দিয়ে প্রতিটি sentence relevant কিনা চেক করে (YES/NO)
- শুধু relevant sentences রেখে `refined_context` তৈরি করে

🟡 Node 6: `generate`

- `refined_context` আর `question` দিয়ে final answer তৈরি করে

Routing Logic

```
def route_after_eval(state) -> str:
    if state['verdict'] == "CORRECT":
        return 'refine'          # সরাসরি refine এ যাও
    else:
        return "rewrite_query"   # INCORRECT বা AMBIGUOUS দুটোই web search এ যাবে
```

৩. 📐 Math / Theory

Cosine Similarity (Vector Store এর ভিতরে)

যখন `retriever.invoke(question)` চলে, তখন ভিতরে হয়:

$$\text{similarity}(Q, D) = (Q \cdot D) / (|Q| \times |D|)$$

- **Q** = Question-এর embedding vector

- **D** = Document chunk-এর embedding vector
- $\cdot$ = Dot product
- ফলাফল 0 (অমিল) থেকে 1 (পুরো মিল)

Relevance Score (doc_eval_node)

LLM কে একটি score দিতে বলা হয়:

Score $\in [0.0, 1.0]$

যদি score > upper_th (0.7): → CORRECT verdict

যদি score < lower_th (0.3): → INCORRECT candidate

সব score < lower_th: → INCORRECT verdict

অন্যথা: → AMBIGUOUS verdict

Manual Calculation উদাহরণ

ধরো 4টি document-এর score এলো: [0.85, 0.40, 0.25, 0.60]

upper_th = 0.7, lower_th = 0.3

any(s > 0.7)? → 0.85 > 0.7 → YES

→ verdict = "CORRECT"

good_docs = যেসব score > lower_th (0.3):

= score 0.85 ✓, 0.40 ✓, 0.25 ✗, 0.60 ✓

= [doc1, doc2, doc4]

আরেকটি উদাহরণ: [0.10, 0.20, 0.15, 0.05]

any(s > 0.7)? → NO

all(s < 0.3)? → 0.10 ✓, 0.20 ✓, 0.15 ✓, 0.05 ✓ → YES

→ verdict = "INCORRECT"

good_docs = []

8. Code-এর ব্যাখ্যা (Python)

নোট: নিচে তোমার `test-2.ipynb` ফাইলের **হুবহু কোড**, শুধু প্রতিটি লাইনে বাংলায় *comment* যোগ করা হয়েছে।

Cell 1: Imports

```
from typing import List, TypedDict, Annotated # Type hints এর জন্য
import re # Regular Expression – text processing এর জন্য
import os # Environment variable পড়ার জন্য

from langchain_community.document_loaders import PyPDFLoader # PDF থেকে document load
from langchain_community.vectorstores import FAISS # Facebook-এর fast vector
from langchain_text_splitters import RecursiveCharacterTextSplitter # Document ছোট chunk-এ
from langchain_core.documents import Document # Document object struct
from langchain_core.prompts import ChatPromptTemplate # LLM-কে prompt দেওয়ার ত
from langchain_core.output_parsers import StrOutputParser # LLM output কে string এ

from langgraph.graph import StateGraph, START, END # LangGraph দিয়ে workflow/graph তৈরির জ
from dotenv import load_dotenv # .env ফাইল থেকে API keys লোড করতে
from pydantic import BaseModel, Field # Structured output define করার জন্য

load_dotenv() # .env ফাইল থেকে GROQ_API_KEY, TAVILY_API_KEY ইত্যাদি লোড হবে
```

Cell 2: Models & Libraries

```
from langchain_huggingface import HuggingFaceEmbeddings # Text কে vector (number) এ রূপান্তর
from langchain_groq import ChatGroq # Groq cloud-এর fast LLM API
import os
from langchain_core.output_parsers import StrOutputParser
```

Cell 3-5: PDF Load & Split

```
# Chunk size 900 character, overlap 100 character দিয়ে splitter তৈরি
# overlap থাকলে দুটি chunk-এর মাঝে তথ্য হারায় না
splitter = RecursiveCharacterTextSplitter(chunk_size=900, chunk_overlap=100)

# book1.pdf ফাইলটি load করো – প্রতিটি page আলাদা Document হবে
docs = PyPDFLoader("./documents/book1.pdf").load()

# প্রতিটি page-কে 900 char chunk-এ ভাগ করো
split_docs = splitter.split_documents(docs)
```

Cell 6-8: Embedding ও LLM Setup

```
# HuggingFace-এর free embedding model – text কে 384-dimension vector এ রূপান্তর করে
embed_model = HuggingFaceEmbeddings(model="sentence-transformers/all-MiniLM-L6-v2")

# Groq-এর Llama 3.1 8B model – fast এবং free tier available
model = ChatGroq(
    model="llama-3.1-8b-instant",
    api_key=os.getenv("GROQ_API_KEY") # .env থেকে API key নাও
)

# FAISS vector store তৈরি করো – সব split_docs কে embed করে index করে রাখে
vector_store = FAISS.from_documents(split_docs, embed_model)
```

Cell 9: Retriever

```
# vector_store কে retriever হিসেবে configure করো
# search_type="similarity" → cosine similarity দিয়ে খুঁজবে
# k=4 → সবচেয়ে মিলে এমন Top-4 document return করবে
retriever = vector_store.as_retriever(search_type="similarity", search_kwargs={'k': 4})
```

Cell 10: `chunks_splitter` Function

```

# এই function একটি বড় text-কে ছোট meaningful sentence-chunks এ ভাঙে
def chunks_splitter(text: str) -> list[str]:
    text = re.sub(r"\s+", " ", text).strip() # একাধিক space/newline কে একটি space এ রূপান্তর

    # বাক্যের শেষ (., !, ?) দিয়ে ভাগ করো
    # (?<=[.!?]) -> lookbehind: .!? এর পরে আসা whitespace খুঁজবে
    raw_sentences = re.split(r"(?<=[.!?])\s+", text)

    chunks = [] # চূড়ান্ত chunk list
    current_chunk = "" # এখন পর্যন্ত তৈরি হওয়া chunk

    for s in raw_sentences:
        s = s.strip() # sentence-এর আগে-পরের space সরাও
        if not s:
            continue # খালি sentence skip করো

        # যদি current chunk + নতুন sentence > 500 char হয়, আগেরটা save করো
        if len(current_chunk) + len(s) + 1 > 500 and current_chunk:
            chunks.append(current_chunk.strip()) # পুরো chunk save
            current_chunk = s # নতুন chunk শুরু
        else:
            # এখনো জায়গা আছে, sentence যোগ করো
            current_chunk = (current_chunk + " " + s).strip()

    # loop শেষে যা বাকি থাকবে সেটাও রাখো
    if current_chunk.strip():
        chunks.append(current_chunk.strip())

    return chunks # সব chunk-এর list return করো

```

Cell 11: Threshold

```

upper_th = 0.7 # এর উপরে score হলে -> "CORRECT" (document ভালো)
lower_th = 0.3 # এর নিচে score হলে -> "INCORRECT" candidate (document খারাপ)

```

Cell 12: State Definition


```

class State(TypedDict):
    question: str          # User-এর input প্রশ্ন
    docs: list[Document]   # retrieve করা raw documents
    ans: str                # intermediate answer (এখানে ব্যবহার হয়নি সরাসরি)

    all_strips: list[str]  # সব sentence chunks (refine_node থেকে)
    kept_strips: list[str] # filter পরে যেগুলো relevant
    refined_context: str   # চূড়ান্ত context যা generate_node পাবে
    good_docs: list[Document] # score > lower_th এমন documents
    verdict: str           # "CORRECT" / "INCORRECT" / "AMBIGUOUS"
    reason: str            # verdict কেন দেওয়া হলো তার ব্যাখ্যা

    web_docs: list[Document] # web search থেকে আসা documents
    rewrite_query: str       # web search এর জন্য rewritten query

    answer: str # চূড়ান্ত উত্তর

```

Cell 13: `retrieve_node`

```

def retrieve_node(state):
    q = state['question']          # state থেকে question নাও
    return {'docs': retriever.invoke(q)} # FAISS থেকে Top-4 docs return করো

```

Cell 14: `doc_eval_node`

```

class DocEvalScore(BaseModel):
    score: float # 0.0 থেকে 1.0 এর মধ্যে relevance score
    reason: str # কেন এই score দেওয়া হলো তার কারণ

# LLM-কে system prompt দাও: একটি chunk ও question দেখে score দাও
doc_eval_prompt = ChatPromptTemplate.from_messages(
    [
        ('system', "You are a strict retrieval evaluator for RAG.\n"
            "You will be given ONE retrieved chunk and a question.\n"
            "Return a relevance score in [0.0, 1.0].\n"
            "- 1.0: chunk alone is sufficient to answer fully/mostly\n"
            "- 0.0: chunk is irrelevant\n"
            "Be conservative with high scores.\n"
            "Also return a short reason.\n"
            "You MUST output valid JSON only."
            "1. \"score\" (a float between 0.0 and 1.0)\n"
            "2. \"reason\" (a short string explaining the score)"
        ),
        ('human', 'Question :{question}\n\nchunk:\n{chunk}') # human turn এ question ও chunk
    ]
)

# doc_eval_prompt → model (JSON mode) → DocEvalScore object
# json_mode মানে LLM কে force করা হচ্ছে valid JSON output দিতে
doc_eval_chain = doc_eval_prompt | model.with_structured_output(DocEvalScore, method='json_mode')

def doc_eval_node(state: State) -> State:
    q = state['question'] # question নাও

    scores: list[float] = [] # প্রতিটি document এর score রাখবে
    reasons: list[str] = [] # প্রতিটি score এর কারণ রাখবে
    good: list[Document] = [] # lower_th এর উপরে score পাওয়া documents

    for d in state['docs']: # প্রতিটি retrieved document এর জন্য
        # LLM দিয়ে evaluate করো → DocEvalScore object পাবে
        out = doc_eval_chain.invoke({'question': q, 'chunk': d.page_content})
        scores.append(out.score) # score সংগ্রহ করো
        reasons.append(out.reason) # reason সংগ্রহ করো

    if out.score > lower_th: # যদি score 0.3 এর উপরে হয়
        good.append(d) # এটাকে "good" document হিসেবে রাখো

```

```
# Verdict 1: যদি কোনো একটি score > 0.7 হয়
if any(s > upper_th for s in scores):
    return {
        'good_docs': good,
        'verdict': 'CORRECT', # সরাসরি refine এ যাবে
        'reason': f'At least one retrieved chunk scored > {upper_th}.'
    }

# Verdict 2: যদি সব score < 0.3 হয়
if len(scores) > 0 and all(s < lower_th for s in scores):
    why = 'No chunk was sufficient'
    return {
        'good_docs': [],
        'verdict': "INCORRECT", # web search করতে হবে
        "reason": f"All retrieved chunks scored < {lower_th}. {why}"
    }

# Verdict 3: মিশ্র scores – AMBIGUOUS
why = 'Mixed relevance signals'
return {
    'good_docs': good,
    'verdict': "AMBIGUOUS", # web search করবে কিন্তু good_docs ও রাখবে
    'reason': f"No chunk scored > {upper_th}, but not all were < {lower_th}. {why}",
}
```

Cell 15: `filter_chain`

```
filter_prompt = ChatPromptTemplate(
    [
        ("system",
         "You are a strict relevance filter. \n"
         "Does the sentence directly help answer the question? \n"
         "Reply with ONLY one word: YES or NO. Nothing else." # কঠোরভাবে YES বা NO শুধু
        ),
        ('human', 'question:{question}\n\nsentence:{sentence}'))
    ]
)

# filter_prompt → model → StrOutputParser (string) → lambda (True/False)
# "YES" থাকলে True, না থাকলে False
filter_chain = filter_prompt | model | StrOutputParser() | (lambda x: "YES" in x.strip().upper())
```

Cell 16: refine_node

```

def refine_node(state: State) -> State:
    q = state['question']
    good_docs = state.get('good_docs', []) # KeyError এড়াতে .get() ব্যবহার
    web_docs = state.get('web_docs', []) # KeyError এড়াতে .get() ব্যবহার
    all_split_chunks = [] # সব sentence chunk
    all_strips = [] # filter এর জন্য সব strips
    all_output = [] # filter এর output (True/False)
    kept_strips = [] # relevant strips যা রাখা হবে
    all_input_for_filter = [] # filter_chain.batch() এর জন্য input list

    # Verdict অনুযায়ী কোন documents ব্যবহার করবে তা ঠিক করো
    if state.get('verdict') == "CORRECT":
        final_docs = good_docs # শুধু good_docs ব্যবহার করো
    elif state.get('verdict') == "INCORRECT":
        final_docs = web_docs # শুধু web_docs ব্যবহার করো
    else:
        # AMBIGUOUS: দুটোই মিলিয়ে ব্যবহার করো
        final_docs = state['good_docs'] + state['web_docs']

    for doc in final_docs:
        # প্রতিটি document-কে ছোট sentence chunk এ ভাগাও
        strips = chunks_splitter(doc.page_content)
        all_split_chunks.extend(strips)

        for s in strips:
            # filter_chain এর জন্য input prepare করো
            all_input_for_filter.append({'question': q, 'sentence': s})
            all_strips.append(s)

    # সব sentence একসাথে batch করে filter করো (API call কম হবে)
    all_output = filter_chain.batch(all_input_for_filter)

    # True হলে রাখো, False হলে বাদ দাও
    for s, r in zip(all_strips, all_output):
        if r:
            kept_strips.append(s)

    # চূড়ান্ত context তৈরি করো – kept strips গুলো দুটি newline দিয়ে জোড়া লাগাও
    refined_context = "\n\n".join(kept_strips)

    return {
        'kept_strips': kept_strips,

```

```
'refined_context': refined_context,  
"all_strips": all_strips  
}
```

Cell 17: `rewrite_query_node`  `web_search_node`

```

load_dotenv()
from langchain_community.tools import TavilySearchResults # Tavily web search tool

class WebQuery(BaseModel):
    query: str # web search এর জন্য rewritten query

rewrite_prompt = ChatPromptTemplate(
    [
        ("system",
         "Rewrite the user question into a web search query composed of keywords. \n"
         "Rules: \n"
         "- Keep it short (5-10 words). \n"          # 5 থেকে 10 word এর মধ্যে রাখো
         "- If the question implies recency, add a constraint like (last 30 days).\n"
         "- Do NOT answer the question. \n"
         "- Return JSON with a single key: query", # JSON format এ return করো
        ),
        ("human", "Question: {question}")
    ]
)

# rewrite_prompt → model (JSON mode) → WebQuery object
rewrite_chain = rewrite_prompt | model.with_structured_output(WebQuery, method='json_mode')

# Query Rewrite Node
def rewrite_query_node(state: State) -> State:
    out = rewrite_chain.invoke({'question': state['question']}) # query rewrite করো
    return {
        'rewrite_query': out.query # rewritten query state এ রাখো
    }

# Tavily search tool – max 2 result আনবে
tavily = TavilySearchResults(max_results=2)

def web_search_node(state: State) -> State:
    q = state['rewrite_query'] # rewritten query নাও
    result = tavily.invoke({'query': q}) # Tavily দিয়ে web search করো

    web_docs = []
    for r in result or []: # result খালি হলেও error না হওয়ার জন্য `or []`
        title = r.get('title', '')
        url = r.get('url', '')
        content = r.get('content', '') or r.get('snippet', '') # content না থাকলে snippet

```

```

# সব তথ্য একটি formatted text এ রাখো
text = f'TITLE: {title}\nURL: {url}\nCONTENT: {content}'

# Document object তৈরি করো
web_docs.append(Document(page_content=text, metadata={'url': url, 'title': title}))

return {
    'web_docs': web_docs # web documents state এ রাখো
}

```

Cell 18: generate

```

answer_prompt = ChatPromptTemplate.from_messages(
    [
        ('system', "Answer only from the context. If not in context, say you don't know"),
        ('human', "Question : {question}\n\nContext:\n{context}")
    ]
)

# answer_prompt → model → StrOutputParser (plain string)
generate_chain = answer_prompt | model | StrOutputParser()

def generate(state):
    question = state['question'] # প্রশ্ন নাও
    context = state['refined_context'] # refine_node থেকে আসা context নাও
    answer = generate_chain.invoke({'question': question, 'context': context}) # উত্তর তৈরি
    return {'answer': answer} # state এ answer রাখো

```

Cell 19: Routing Function

```

def route_after_eval(state: State) -> str:
    if state['verdict'] == "CORRECT":
        return 'refine' # CORRECT হলে সরাসরি refine_node এ পাঠাও
    else:
        return "rewrite_query" # INCORRECT বা AMBIGUOUS হলে query rewrite করো

```


Cell 20: Graph তৈরি ও Compile

```

g = StateGraph(State) # State TypedDict দিয়ে graph তৈরি করো

# Nodes যোগ করো
g.add_node("retrieve", retrieve_node)
g.add_node('eval_each_doc', doc_eval_node)
g.add_node('rewrite_query', rewrite_query_node)
g.add_node('web_search', web_search_node)
g.add_node("refine", refine_node)
g.add_node("generate", generate)

# Edges (connections) যোগ করো
g.add_edge(START, "retrieve") # শুরু → retrieve_node
g.add_edge("retrieve", "eval_each_doc") # retrieve → evaluate

# Conditional Edge: verdict দেখে route করো
g.add_conditional_edges(
    "eval_each_doc", # এই node থেকে
    route_after_eval, # এই function দিয়ে decide করো
    {"refine": "refine", "rewrite_query": "rewrite_query"} # return value → target node map
)

g.add_edge('rewrite_query', 'web_search') # query rewrite → web search
g.add_edge('web_search', 'refine') # web search → refine
g.add_edge("refine", "generate") # refine → generate
g.add_edge("generate", END) # generate → শেষ

app = g.compile() # graph compile করো – এখন invoke করার জন্য ready
print("Graph compiled successfully!")

```

Cell 21: Invoke

```
result = app.invoke({  
    "question": "What is neural network?"  
})
```

Expected Output (CORRECT verdict হলে):

VERDICT: CORRECT

REASON: At least one retrieved chunk scored > 0.7.

Answer: In the context of the provided text, a neural network is...

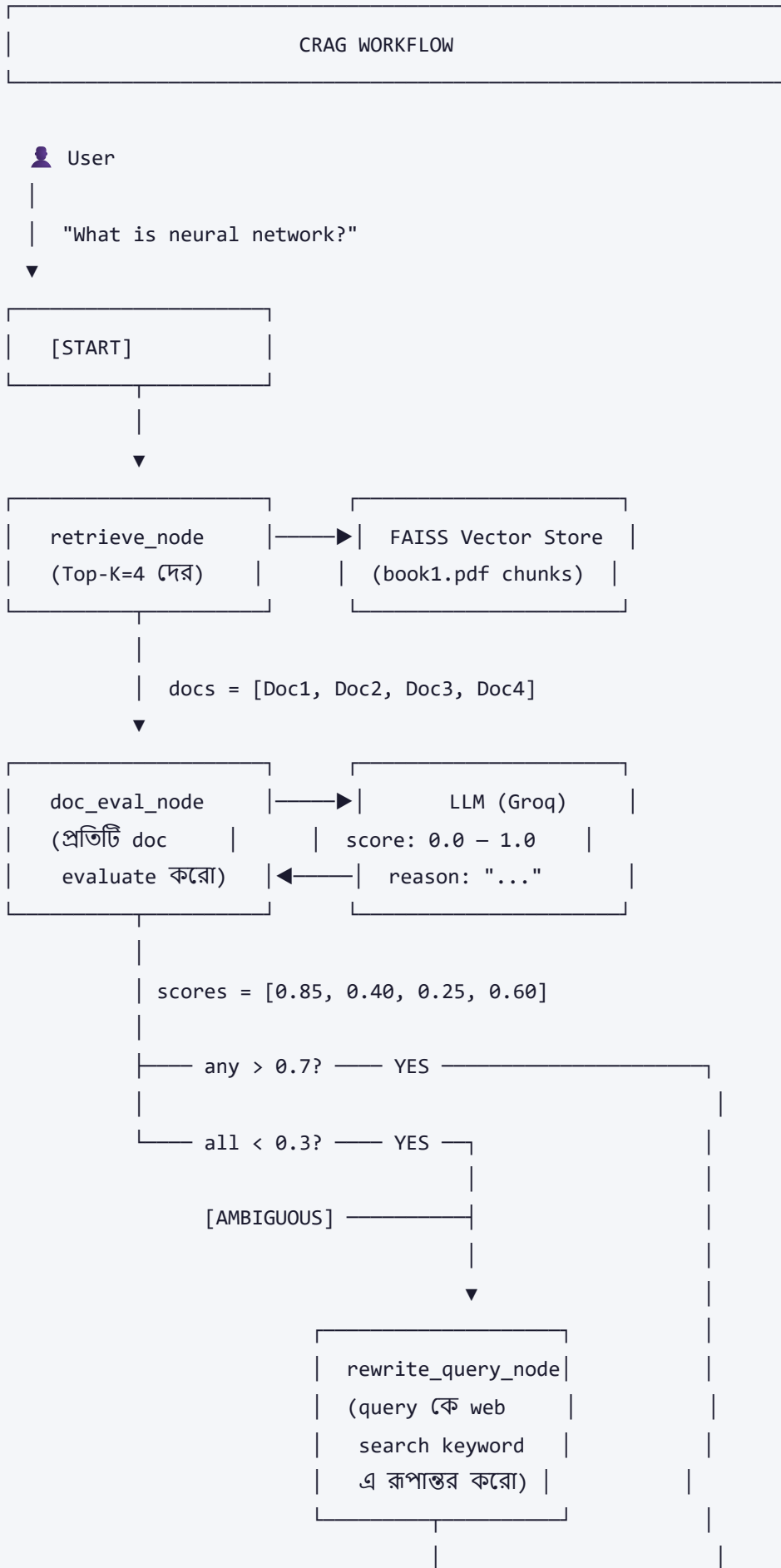
```
print("VERDICT: ", result['verdict'])
```

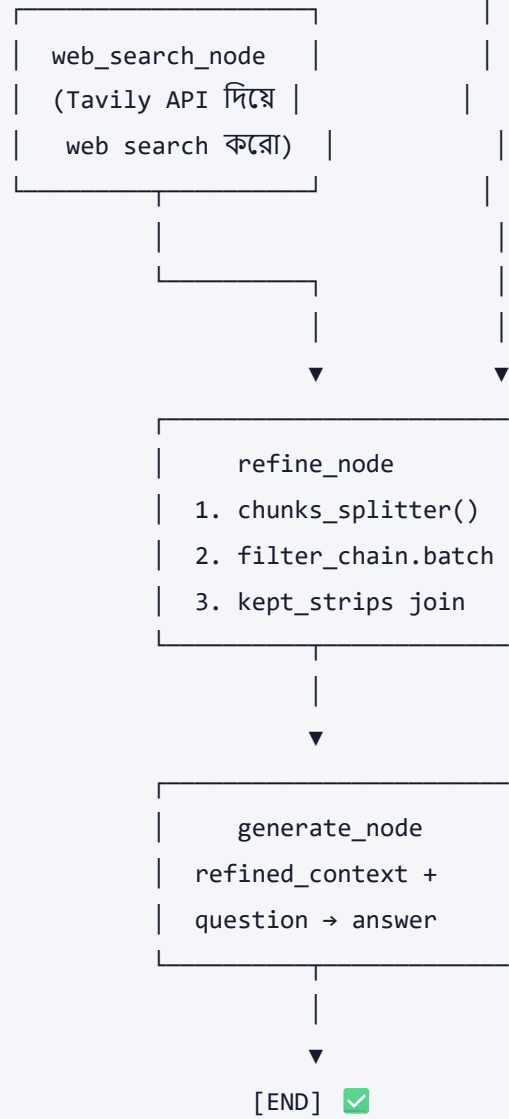
```
print("REASON: ", result['reason'])
```

```
print("Answer:", result["answer"])
```

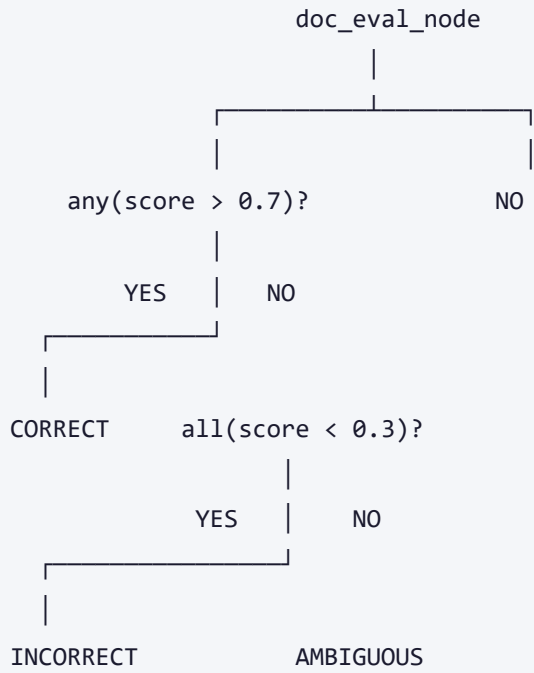
🔗 🎨 Visual / Diagram

CRAG Architecture Diagram





Verdict Decision Tree



৬. Real-world Use Cases

১. Legal Document QA System

আইনি দলিলের প্রশ্নোত্তর সিস্টেম:

- পুরনো আইনের বই PDF ব্যবহার করে retrieve করে
- নতুন আইন বা amendment এর প্রশ্নে INCORRECT → web search করে সর্বশেষ তথ্য আনে

২. Medical Knowledge Base

চিকিৎসা সংক্রান্ত প্রশ্নে:

- Hospital-এর internal documents থেকে retrieve
- নতুন drug বা treatment এর প্রশ্নে web থেকে WHO/PubMed query করে

৩. Corporate Internal Chatbot

কোম্পানির HR/Finance policy chatbot:

- Company-এর PDF documents থেকে retrieve
- Policy update হলেও সর্বশেষ তথ্য web থেকে নিশ্চিত করে

8. 🎓 Educational Q&A Platform

ছাত্রদের পড়াশুনার সহায়তায়:

- Textbook PDF থেকে context নেয়
- প্রশ্নের উত্তর না থাকলে Wikipedia বা MIT OpenCourseWare থেকে আনে

৫. 📖 Research Assistant

গবেষকদের সাহায্যে:

- Research paper PDF index করা
- সর্বশেষ publication এর জন্য ArXiv বা Google Scholar query করে

কোন Company ব্যবহার করে?

Company	ব্যবহার
LangChain	Official CRAG tutorial ও template আছে
Cohere	Command R+ model এ RAG Grounding
Anthropic	Claude-এর tool use এর সাথে CRAG pattern
Microsoft	Azure AI Search এ CRAG-like evaluation

৭. ⚖️ Pros & Cons

সুবিধা 	অসুবিধা 
ভুল document detect করে সংশোধন করে	LLM evaluation call বেশি → বেশি cost
Web search দিয়ে fresh information আনে	Tavily API key দরকার (paid)
Sentence-level filtering — noise কমায়	Latency বেশি (multiple LLM calls)
AMBIGUOUS case handle করে gracefully	Rate limit issue হতে পারে
Standard RAG এর চেয়ে accurate answer	Web search always reliable না
LangGraph দিয়ে graph clear ও maintainable	Setup complex

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: `max_result` বনাম `max_results`

```
# ❌ ভুল – চলবে কিন্তু কোনো effect নেই
tavily = TavilySearchResults(max_result=2)

# ✅ সঠিক
tavily = TavilySearchResults(max_results=2)
```

ভুল ২: State key missing হলে `KeyError`

```
# ❌ ভুল – যদি web_docs state এ না থাকে তাহলে crash
web_docs = state['web_docs']

# ✅ সঠিক – default হিসেবে [] দিলে safe
web_docs = state.get('web_docs', [])
```

ভুল ৩: `all()` empty list এ সবসময় `True` দেয়

```
# যদি docs = [] হয় তাহলে scores = []
# all(s < lower_th for s in []) → True ← বিপজ্জনক!
# তাই len(scores) > 0 condition দিতে হবে
if len(scores) > 0 and all(s < lower_th for s in scores):
    ...
```

ভুল ৪: `rewrite_prompt` এ word count typo

```
# ❌ ভুল – "614 words" কোনো অর্থ রাখে না
"- Keep it short (614 words). \n"

# ✅ সঠিক
"- Keep it short (5-10 words). \n"
```

ভুল ৫: Batch vs Single call

```
# ❌ ধীর – প্রতিটি sentence এর জন্য আলাদা API call
for s in all_strips:
    output = filter_chain.invoke(...)

# ✅ দ্রুত – একসাথে batch করে পাঠাও
all_output = filter_chain.batch(all_input_for_filter)
```

ভুল ৬: Route function এ AMBIGUOUS miss করা

❌ ভুল – AMBIGUOUS handle করা ভুললে CORRECT পাথে যাবে

```
def route(state):
    if state['verdict'] == "CORRECT":
        return 'refine'
    if state['verdict'] == "INCORRECT":
        return 'rewrite_query'
    # AMBIGUOUS হলে কী হবে? → None → Error!
```

✅ সঠিক – else দিয়ে সব cover করে

```
def route(state):
    if state['verdict'] == "CORRECT":
        return 'refine'
    else: # INCORRECT এবং AMBIGUOUS দুটোই
        return "rewrite_query"
```

৯. 🔗 Related Topics

আগে কী জানা দরকার?

1. **Basic RAG** — Vector Store, Embedding, Retrieval বুঝতে হবে
2. **LangChain Basics** — Chain, Prompt, Parser জানতে হবে
3. **LangGraph** — StateGraph, Node, Edge concept বুঝতে হবে
4. **Pydantic** — BaseModel, structured output বুঝতে হবে
5. **Vector Database** — FAISS বা ChromaDB কীভাবে কাজ করে

পরে কী শেখা উচিত?

1. **Self-RAG** — আরো advanced, নিজেই critique করে
2. **Agentic RAG** — Multiple tools ব্যবহার করে
3. **GraphRAG** — Knowledge Graph দিয়ে RAG
4. **HyDE (Hypothetical Document Embeddings)** — Better retrieval
5. **Re-ranking** — Retrieved documents-কে score দিয়ে সাজানো
6. **Multi-hop RAG** — একাধিক step এ answer করা

১০. 🗨️ Memory Tricks

মনে রাখার সহজ কৌশল

C-R-A-G মনে রাখার Formula:

C → Check করো (doc_eval_node: score দাও)
R → Reject করো (INCORRECT/AMBIGUOUS হলে Web এ যাও)
A → Augment করো (refine_node: relevant sentence রাখো)
G → Generate করো (generate: final answer দাও)

Verdict মনে রাখার কৌশল

score > 0.7 → CORRECT → "বই থেকেই পাওয়া যাবে ✅"
score < 0.3 → INCORRECT → "বই কাজে লাগবে না ❌"
মারামারি → AMBIGUOUS → "নিশ্চিত না, দুটো দেখি 🤔"

১ লাইনে সারসংক্ষেপ

"CRAG হলো এমন একটি RAG সিস্টেম যেটি নিজেই বুঝতে পারে retrieve করা document কতটা ভালো — ভালো না হলে web থেকে নতুন তথ্য এনে সঠিক উত্তর দেয়।"

📅 তৈরির তারিখ: ২০২৬-০৪-০৯ | 📁 Source: test-2.ipynb (CRAG Implementation)

📚 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

🗨️ AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes